



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

КБ-3 «Разработка программных решений и системное программирование»

Отчет по практической работе №5

«Проект каталога: представления и тестирование»

по дисциплине

«Сборка, тестирование и верификация программного продукта»

Выполнил студент группы БСБО-09-23

Фёдоров Антон Сергеевич

Работа выполнена

«07» мая 2026 г.

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ЦЕЛЬ И ПЛАН РАБОТЫ	3
2. ОПИСАНИЕ ПРЕДСТАВЛЕНИЙ И GET-ПАРАМЕТРОВ	4
3. ТЕСТ-ПЛАН И МАТРИЦА ТЕСТ-КЕЙСОВ.....	6
4. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ	9
5. ЧЕК-ЛИСТ ПО ТРЕБОВАНИЯМ	11
6. ИСХОДНЫЙ КОД	12
6.1. Представления.....	12
6.2. Маршрутизация.....	13
6.3. HTML-шаблоны	14
6.4. Тесты маршрутов	17
6.5. Тесты контента.....	18
ВЫВОДЫ	22

1. ЦЕЛЬ И ПЛАН РАБОТЫ

Цель: реализовать этап проекта каталога товаров на Django, связанный с проверкой представлений: доработать вывод списка товаров, добавить GET-сортировку по обязательному, необязательному и типовому полям, проверить маршруты, шаблоны, контекст, фильтрацию и сортировку автоматизированными тестами.

План:

1. Изучить требования к практической работе 5.
2. Использовать проект каталога аккумуляторов с моделями базы данных как основу.
3. Доработать представление списка товаров для обработки параметра sort.
4. Добавить варианты сортировки по названию, цене, весу и типу аккумулятора.
5. Сохранить поиск, фильтрацию по типу и фильтрацию по наличию через GET-параметры.
6. Передать в шаблон выбранную сортировку и список вариантов сортировки.
7. Обновить HTML-шаблон списка товаров, добавив поле выбора сортировки.
8. Перенести тесты в пакет catalog/tests и разделить их по назначению.
9. Подготовить TEST_PLAN.md с обязательными разделами и матрицей тест-кейсов.
10. Запустить тесты и coverage, затем оформить отчет.

2. ОПИСАНИЕ ПРЕДСТАВЛЕНИЙ И GET-ПАРАМЕТРОВ

Тема каталога: аккумулятор.

Проект Django находится в папке ПЗ5/catalog_project. В работе используются модели BatteryType и Battery, созданные на этапе базы данных. BatteryType хранит тип аккумулятора, а Battery хранит конкретный товар с артикулом, названием, брендом, типом, емкостью, напряжением, ценой, остатком, гарантией и дополнительными характеристиками.

Основная доработка ПЗ5 выполнена в представлении product_list. Страница списка товаров открывается по адресу / и получает данные из базы через Battery.objects.select_related('battery_type'). Представление обрабатывает GET-параметры:

- q - поиск по названию, бренду, артикулу и описанию;
- type - фильтр по типу аккумулятора;
- availability - фильтр по наличию товара;
- sort - выбранный способ сортировки.

Для сортировки создан словарь SORT_OPTIONS. В нем описаны четыре варианта:

- name - сортировка по названию;
- price - сортировка по обязательному полю цены;
- weight - сортировка по необязательному полю веса;
- type - сортировка по названию типа аккумулятора.

Если в GET-парамetre sort передано неизвестное значение, представление возвращается к сортировке по умолчанию name. Это защищает страницу от некорректных параметров и делает поведение каталога предсказуемым.

В шаблон product_list.html передаются products, battery_types, query, selected_type, selected_availability, selected_sort и sort_options. Благодаря этому

страница может показать найденные товары, сохранить выбранные значения в фильтрах и отобразить текущий способ сортировки.

Страница детального описания товара открывается по адресу /product/<int:pk>/. Она получает товар через get_object_or_404 и возвращает 404, если записи с указанным pk нет.

Страница /about/ является статической страницей о магазине аккумуляторов. В ПЗ5 она также включена в тесты, потому что методичка требует проверять не только каталог, но и статическую страницу About.

Скриншоты страниц сохранены в папке ПЗ5/screenshots/:

- product_list.png - список товаров с фильтрами и сортировкой;
- product_detail.png - детальная страница товара.

Рисунок 1. Список товаров каталога аккумуляторов

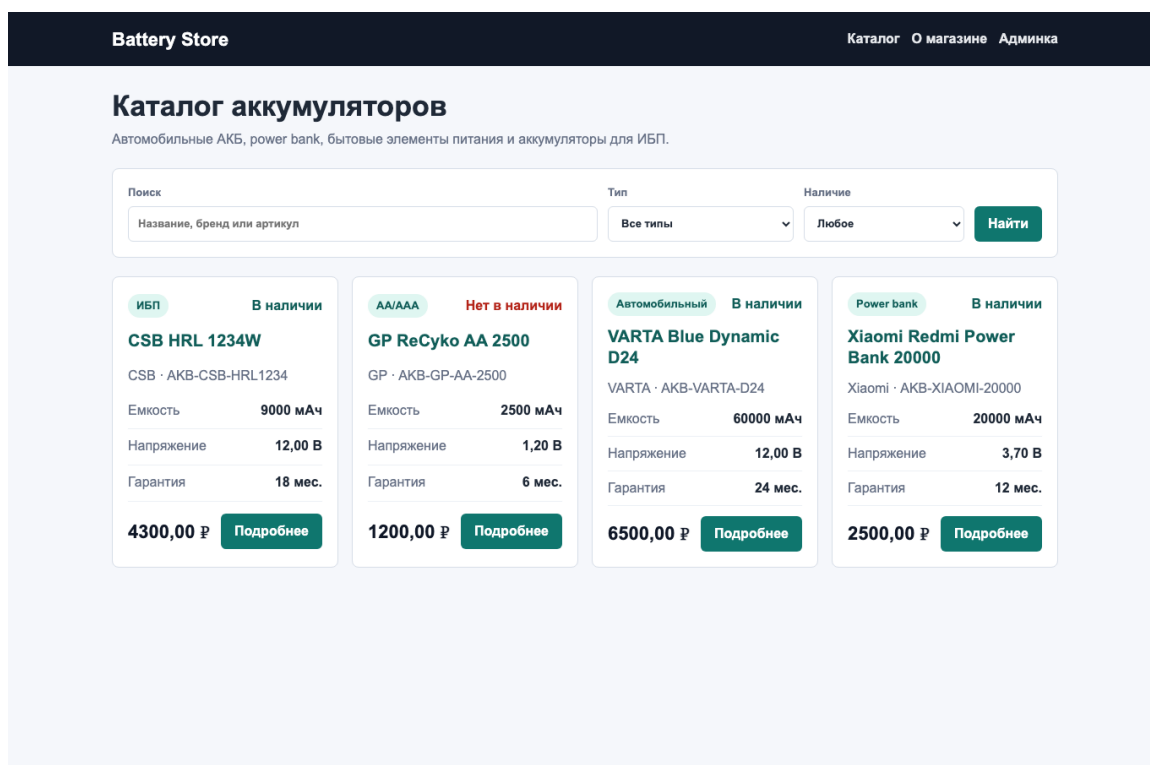


Рисунок 2. Детальная страница товара каталога аккумуляторов

[Назад к каталогу](#)

VARTA Blue Dynamic D24

VARTA · АКВ-VARTA-D24 · Автомобильный

6500,00 ₺

Автомобильный аккумулятор емкостью 60 Ач с пусковым током 540 А.

На складе: 4 шт.

Тип	Автомобильный
Емкость	60000 мАч
Напряжение	12,00 В
Пусковой ток	540 А
Вес	14500 г
Быстрая зарядка	Нет
Большая емкость	Да
Тяжелый	Да
Гарантия	24 мес.

3. ТЕСТ-ПЛАН И МАТРИЦА ТЕСТ-КЕЙСОВ

Для практической работы подготовлен файл TEST_PLAN.md.
Идентификатор тест-плана: ТР-АКВ-2026-05.

Тест-план описывает проверку модуля отображения каталога аккумуляторов. Проверяются view-функции `product_list`, `product_detail` и `about`, а также URL `/`, `/product/<pk>/` и `/about/`.

Проверяемые возможности:

- доступность маршрутов и связь URL с нужными view-функциями;
- GET-статусы 200 и 404;
- загрузка ожидаемых HTML-шаблонов;
- наличие нужных ключей в `response.context`;
- вывод товаров, созданных в `setUpTestData`;
- поиск через GET-параметр `q`;
- фильтрация по типу аккумулятора через GET-параметр `type`;
- фильтрация по наличию через GET-параметр `availability`;
- сортировка по названию;
- сортировка по обязательному полю `price`;
- сортировка по необязательному полю `weight_grams`;
- сортировка по полю типа `battery_type`;
- обработка неверного значения сортировки;
- корректный вывод пустого списка;
- отображение страницы `About`.

На этом этапе не тестируются авторизация, права доступа, POST-запросы, формы создания, формы редактирования, формы удаления, API и работа Django admin.

Подход к тестированию основан на стандартных инструментах Django: `TestCase`, тестовый `Client`, `setUpTestData`, `reverse()`, `resolve()`,

`assertTemplateUsed()`, `assertContains()`, `assertNotContains()` и проверка `response.context`.

Критерий прохождения: тест считается пройденным, если страница возвращает ожидаемый статус, URL связан с нужной view-функцией, загружается правильный шаблон, контекст содержит нужные данные, а результаты фильтрации и сортировки совпадают с ожидаемыми.

Критерии приостановки: тестирование нужно остановить, если не создается тестовая база данных, не применяются миграции, не создаются тестовые данные или приложение не запускается из-за ошибки конфигурации.

К сдаче относятся пакет `catalog/tests`, файлы `test_routes.py` и `test_content.py`, `TEST_PLAN.md`, вывод запуска тестов и отчет покрытия.

Матрица тест-кейсов

- TC-01. Главная страница каталога открывается: GET / возвращает 200 и `view product_list`.
- TC-02. URL главной страницы строится через `reverse("product_list")` и равен `/`.
- TC-03. Список содержит товары из тестовой базы.
- TC-04. Контекст списка содержит `products` и `battery_types`.
- TC-05. Поиск `?q=VARTA` оставляет в списке только товар VARTA.
- TC-06. Фильтрация `?type=aa-aaa` показывает только товары типа AA/AAA.
- TC-07. Фильтрация `?availability=out_of_stock` показывает только товары без остатка.
- TC-08. Поиск несуществующего товара показывает пустое состояние.
- TC-09. Сортировка по умолчанию выполняется по `name`.
- TC-10. Сортировка `?sort=price` выполняется по `price` и `name`.
- TC-11. Сортировка `?sort=weight` выполняется по `weight_grams` и `name`.
- TC-12. Сортировка `?sort=type` выполняется по `battery_type__name` и `name`.

- TC-13. Неверная сортировка ?sort=wrong заменяется сортировкой name.
- TC-14. Детальная страница товара возвращает 200, нужный шаблон и context product.
- TC-15. Несуществующий товар возвращает 404.
- TC-16. Страница About открывается с кодом 200.
- TC-17. About использует шаблон about.html и содержит текст о магазине.
- TC-18. Фильтрация и сортировка работают совместно:
availability=in_stock&sort=price.

4. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Для проверки проекта использовался встроенный тестовый фреймворк Django на базе unittest.

Команда запуска тестов:

```
cd П35/catalog_project
python3 manage.py test catalog.tests --verbosity=2
```

Фактический результат:

```
Found 23 test(s).
Ran 23 tests in 0.027s
OK
```

Всего реализовано 23 теста. В файле `catalog/tests/test_routes.py` находится 8 тестов маршрутов и статус-кодов. В файле `catalog/tests/test_content.py` находится 15 тестов содержимого, шаблонов, контекста, поиска, фильтрации и сортировки.

Команды проверки покрытия:

```
python3 -m coverage run --source='.' manage.py test catalog.tests
python3 -m coverage report
```

Фактический результат покрытия:

Name	Stmts	Miss	Cover
-----	-----	-----	-----
catalog/views.py	29	0	100%
catalog/tests/test_content.py	83	0	100%
catalog/tests/test_routes.py	32	0	100%
TOTAL	279	17	94%

В тестах проверяется:

- `reverse` и `resolve` для основных маршрутов;
- GET-запросы к списку, детальной странице и About;
- 404 для несуществующего товара;
- использование шаблонов `product_list.html`, `product_detail.html` и `about.html`;
- наличие `products` и `battery_types` в контексте;
- вывод товаров из базы данных;
- поиск по строке `q`;
- фильтрация по типу;
- фильтрация по отсутствующим товарам;

- пустое состояние списка;
- сортировка по названию, цене, весу и типу;
- fallback на сортировку name при неверном параметре sort;
- совместная работа фильтра availability=in_stock и сортировки sort=price.

5. ЧЕК-ЛИСТ ПО ТРЕБОВАНИЯМ

- [x] Список товаров доработан для сортировки не только по алфавиту.
- [x] Добавлена сортировка по обязательному полю price.
- [x] Добавлена сортировка по необязательному полю weight_grams.
- [x] Добавлена сортировка по полю типа товара battery_type__name.
- [x] Группа товаров показывается через GET-параметр type.
- [x] Поиск выполняется через GET-параметр q.
- [x] Фильтрация по наличию выполняется через GET-параметр availability.
- [x] Выбранная сортировка сохраняется в selected_sort.
- [x] Некорректное значение sort заменяется сортировкой по умолчанию.
- [x] Тесты перенесены в пакет catalog/tests.
- [x] Тесты разделены на test_routes.py и test_content.py.
- [x] Разработан TEST_PLAN.md с обязательными разделами.
- [x] В тест-плане есть матрица из 18 тест-кейсов.
- [x] Проверяются GET-запросы, маршруты, шаблоны, контекст, фильтрация, сортировка и About.
- [x] POST-запросы, формы, авторизация и API не входят в проверку ПЗ5.
- [x] Реализовано 23 автоматизированных теста.
- [x] Все тесты проходят успешно.
- [x] Покрытие catalog/views.py составляет 100%.
- [x] Сделаны скриншоты страниц.

6. ИСХОДНЫЙ КОД

6.1. Представления

Листинг 1. Файл catalog/views.py

```
from django.db.models import Q
from django.shortcuts import get_object_or_404, render

from .models import Battery, BatteryType

SORT_OPTIONS = {
    'name': {
        'label': 'По названию',
        'order_by': ('name',),
    },
    'price': {
        'label': 'По цене',
        'order_by': ('price', 'name'),
    },
    'weight': {
        'label': 'По весу',
        'order_by': ('weight_grams', 'name'),
    },
    'type': {
        'label': 'По типу',
        'order_by': ('battery_type__name', 'name'),
    },
}
DEFAULT_SORT = 'name'

def product_list(request):
    products = Battery.objects.select_related('battery_type')
    query = request.GET.get('q', '').strip()
    selected_type = request.GET.get('type', '').strip()
    selected_availability = request.GET.get('availability', '').strip()
    selected_sort = request.GET.get('sort', DEFAULT_SORT).strip()

    if query:
        products = products.filter(
            Q(name__icontains=query)
            | Q(brand__icontains=query)
            | Q(sku__icontains=query)
            | Q(description__icontains=query)
        )

    if selected_type:
        products = products.by_type(selected_type)

    if selected_availability == 'in_stock':
        products = products.in_stock()
    elif selected_availability == 'out_of_stock':
        products = products.filter(stock_quantity=0)

    if selected_sort not in SORT_OPTIONS:
        selected_sort = DEFAULT_SORT
    products = products.order_by(*SORT_OPTIONS[selected_sort]['order_by'])

    context = {
        'products': products,
        'battery_types': BatteryType.objects.all(),
        'query': query,
        'selected_type': selected_type,
        'selected_availability': selected_availability,
        'selected_sort': selected_sort,
        'sort_options': SORT_OPTIONS,
    }
    return render(request, 'catalog/product_list.html', context)

def product_detail(request, pk):
    product = get_object_or_404(Battery.objects.select_related('battery_type'), pk=pk)
    return render(request, 'catalog/product_detail.html', {'product': product})
```

```
def about(request):
    return render(request, 'catalog/about.html')
```

6.2. Маршрутизация

Листинг 2. Файл catalog_project/urls.py

```
"""
URL configuration for catalog_project project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/6.0/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('catalog.urls')),
]
```

Листинг 3. Файл catalog/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.product_list, name='product_list'),
    path('product/<int:pk>/', views.product_detail, name='product_detail'),
    path('about/', views.about, name='about'),
]
```

6.3. HTML-шаблоны

Для отображения страниц используются шаблоны из папки catalog/templates/catalog/. Общий шаблон base.html содержит HTML-каркас, меню навигации и CSS-оформление каталога.

Листинг 4. Файл catalog/templates/catalog/product_list.html

```
{% extends 'catalog/base.html' %}

{% block title %}Каталог аккумуляторов{% endblock %}

{% block content %}
<section class="page-header">
    <div>
        <h1>Каталог аккумуляторов</h1>
        <p>Автомобильные АКБ, power bank, бытовые элементы питания и аккумуляторы для ИБП.</p>
    </div>
</section>

<form class="filters" method="get">
    <label>
        Поиск
        <input type="search" name="q" value="{{ query }}" placeholder="Название, бренд или артикул">
    </label>
</form>
```

```

<label>
    Тип
    <select name="type">
        <option value="">Все типы</option>
        {% for battery_type in battery_types %}
            <option value="{{ battery_type.slug }}" {% if selected_type == battery_type.slug %}
selected{% endif %}>
                {{ battery_type.name }}
            </option>
        {% endfor %}
    </select>
</label>
<label>
    Наличие
    <select name="availability">
        <option value="">Любое</option>
        <option value="in_stock" {% if selected_availability == 'in_stock' %} selected{% endif
%}>В наличии</option>
        <option value="out_of_stock" {% if selected_availability == 'out_of_stock' %} selected{%
endif %}>Нет в наличии</option>
    </select>
</label>
<label>
    Сортировка
    <select name="sort">
        {% for sort_value, sort_option in sort_options.items %}
            <option value="{{ sort_value }}" {% if selected_sort == sort_value %} selected{%
endif %}>
                {{ sort_option.label }}
            </option>
        {% endfor %}
    </select>
</label>
<button class="button" type="submit">Найти</button>
</form>

{% if products %}
    <section class="grid">
        {% for product in products %}
            <article class="product-card">
                <div class="product-card_top">
                    <span class="badge">{{ product.battery_type.name }}</span>
                    {% if product.stock_quantity > 0 %}
                        <span class="status">В наличии</span>
                    {% else %}
                        <span class="status status--empty">Нет в наличии</span>
                    {% endif %}
                </div>
                <h2><a href="{{ product.get_absolute_url }}">{{ product.name }}</a></h2>
                <p>{{ product.brand }} · {{ product.sku }}</p>
                <dl class="specs">
                    <div>
                        <dt>Емкость</dt>
                        <dd>{{ product.capacity_mah }} мАч</dd>
                    </div>
                    <div>
                        <dt>Напряжение</dt>
                        <dd>{{ product.voltage_v }} В</dd>
                    </div>
                    <div>
                        <dt>Гарантия</dt>
                        <dd>{{ product.warranty_months }} мес.</dd>
                    </div>
                </dl>
                <div class="meta-row">
                    <span class="price">{{ product.price }} Р</span>
                    <a class="button" href="{{ product.get_absolute_url }}">Подробнее</a>
                </div>
            </article>
        {% endfor %}
    </section>
{% else %}
    <section class="empty-state">
        <p>По заданным условиям аккумуляторы не найдены.</p>
    </section>
{% endif %}
{% endblock %}

```

Листинг 5. Файл catalog/templates/catalog/product_detail.html

```

{% extends 'catalog/base.html' %}

{% block title %}{{ product.name }}{% endblock %}

```

```

{% block content %}
<a class="back-link" href="{% url 'product_list' %}">Назад к каталогу</a>

<article class="detail-panel">
  <header class="page-header">
    <div>
      <h1>{{ product.name }}</h1>
      <p>{{ product.brand }} · {{ product.sku }} · {{ product.battery_type.name }}</p>
    </div>
    <span class="price">{{ product.price }} Р</span>
  </header>

  <section class="detail-grid">
    <div>
      <p>{{ product.description|default:'Описание пока не добавлено.' }}</p>
      {% if product.stock_quantity > 0 %}
        <p class="status">На складе: {{ product.stock_quantity }} шт.</p>
      {% else %}
        <p class="status status--empty">Сейчас нет в наличии</p>
      {% endif %}
    </div>

    <dl class="specs">
      <div>
        <dt>Тип</dt>
        <dd>{{ product.battery_type.name }}</dd>
      </div>
      <div>
        <dt>Емкость</dt>
        <dd>{{ product.capacity_mah }} мАч</dd>
      </div>
      <div>
        <dt>Напряжение</dt>
        <dd>{{ product.voltage_v }} В</dd>
      </div>
      <div>
        <dt>Пусковой ток</dt>
        <dd>{% if product.starting_current_a %}{{ product.starting_current_a }} А{% else %}Не указан{% endif %}</dd>
      </div>
      <div>
        <dt>Вес</dt>
        <dd>{% if product.weight_grams %}{{ product.weight_grams }} г{% else %}Не указан{% endif %}</dd>
      </div>
      <div>
        <dt>Быстрая зарядка</dt>
        <dd>{% if product.fast_charge %}Да{% else %}Нет{% endif %}</dd>
      </div>
      <div>
        <dt>Большая емкость</dt>
        <dd>{% if product.is_high_capacity %}Да{% else %}Нет{% endif %}</dd>
      </div>
      <div>
        <dt>Тяжелый</dt>
        <dd>{% if product.is_heavy %}Да{% else %}Нет{% endif %}</dd>
      </div>
      <div>
        <dt>Гарантия</dt>
        <dd>{{ product.warranty_months }} мес.</dd>
      </div>
    </dl>
  </section>
</article>
{% endblock %}

```

Листинг 6. Файл catalog/templates/catalog/about.html

```

{% extends 'catalog/base.html' %}

{% block title %}О магазине аккумуляторов{% endblock %}

{% block content %}
<section class="page-header">
  <div>
    <h1>О магазине</h1>
    <p>Каталог рассчитан на тестовое наполнение через админку Django и быстрый поиск по товарной базе.</p>
  </div>
</section>

```



```

<section class="detail-panel">
    <p>Магазин специализируется на аккумуляторах для автомобиля, портативной техники, бытовых устройств и резервного питания.</p>
    <p>Для каждого товара хранится тип, артикул, бренд, емкость, напряжение, цена, складской остаток, гарантия и дополнительные характеристики.</p>
</section>
{% endblock %}

```

6.4. Тесты маршрутов

Листинг 7. Файл catalog/tests/test_routes.py

```

from django.test import TestCase
from django.urls import resolve, reverse

from catalog import views
from catalog.models import Battery, BatteryType

class CatalogRouteTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        battery_type = BatteryType.objects.create(name='Автомобильный', slug='car')
        cls.product = Battery.objects.create(
            sku='AKB-VARTA-D24',
            name='VARTA Blue Dynamic D24',
            brand='VARTA',
            battery_type=battery_type,
            capacity_mah=60000,
            voltage_v='12.00',
            starting_current_a=540,
            weight_grams=14500,
            price='6500.00',
            stock_quantity=4,
            warranty_months=24,
            description='Автомобильный аккумулятор емкостью 60 Ач с пусковым током 540 А.',
        )

    def test_product_list_url_resolves_to_product_list_view(self):
        match = resolve(reverse('product_list'))

        self.assertEqual(match.func, views.product_list)

    def test_product_detail_url_resolves_to_product_detail_view(self):
        match = resolve(reverse('product_detail', args=[self.product.pk]))

        self.assertEqual(match.func, views.product_detail)

    def test_about_url_resolves_to_about_view(self):
        match = resolve(reverse('about'))

        self.assertEqual(match.func, views.about)

    def test_product_list_reverse_returns_catalog_root(self):
        self.assertEqual(reverse('product_list'), '/')

    def test_product_list_get_returns_200(self):
        response = self.client.get(reverse('product_list'))

        self.assertEqual(response.status_code, 200)

    def test_product_detail_get_returns_200(self):
        response = self.client.get(reverse('product_detail', args=[self.product.pk]))

        self.assertEqual(response.status_code, 200)

    def test_product_detail_missing_product_returns_404(self):
        response = self.client.get(reverse('product_detail', args=[999]))

        self.assertEqual(response.status_code, 404)

    def test_about_get_returns_200(self):
        response = self.client.get(reverse('about'))

        self.assertEqual(response.status_code, 200)

```

6.5. Тесты контента

Листинг 8. Файл catalog/tests/test_content.py

```
from django.test import TestCase
from django.urls import reverse

from catalog.models import Battery, BatteryType

class CatalogContentTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        car = BatteryType.objects.create(
            name='Автомобильный',
            slug='car',
            description='Стартерные аккумуляторы для легковых автомобилей.',
        )
        power_bank = BatteryType.objects.create(
            name='Power bank',
            slug='power-bank',
            description='Портативные внешние аккумуляторы для гаджетов.',
        )
        aa_aaa = BatteryType.objects.create(
            name='AA/AAA',
            slug='aa-aaa',
            description='Бытовые аккумуляторные элементы питания.',
        )
        ups = BatteryType.objects.create(
            name='ИБП',
            slug='ups',
            description='Аккумуляторы для источников бесперебойного питания.',
        )

        Battery.objects.create(
            sku='AKB-VARTA-D24',
            name='VARTA Blue Dynamic D24',
            brand='VARTA',
            battery_type=car,
            capacity_mah=60000,
            voltage_v='12.00',
            starting_current_a=540,
            weight_grams=14500,
            price='6500.00',
            stock_quantity=4,
            warranty_months=24,
            description='Автомобильный аккумулятор емкостью 60 Ач с пусковым током 540 А.',
        )
        Battery.objects.create(
            sku='AKB-XIAOMI-20000',
            name='Xiaomi Redmi Power Bank 20000',
            brand='Xiaomi',
            battery_type=power_bank,
            capacity_mah=20000,
            voltage_v='3.70',
            weight_grams=430,
            price='2500.00',
            stock_quantity=12,
            warranty_months=12,
            fast_charge=True,
            description='Портативный аккумулятор на 20000 мАч с поддержкой быстрой зарядки.',
        )
        Battery.objects.create(
            sku='AKB-GP-AA-2500',
            name='GP ReCyko AA 2500',
            brand='GP',
            battery_type=aa_aaa,
            capacity_mah=2500,
            voltage_v='1.20',
            weight_grams=120,
            price='1200.00',
            stock_quantity=0,
            warranty_months=6,
            description='Комплект из четырех Ni-MH аккумуляторов AA емкостью 2500 мАч.',
        )
        Battery.objects.create(
            sku='AKB-CSB-HRL1234',
            name='CSB HRL 1234W',
            brand='CSB',
            battery_type=ups,
```

```

        capacity_mah=9000,
        voltage_v='12.00',
        weight_grams=2600,
        price='4300.00',
        stock_quantity=3,
        warranty_months=18,
        description='Герметичный свинцово-кислотный аккумулятор для источников бесперебойного
питания.',
    )

    def product_skus(self, response):
        return list(response.context['products'].values_list('sku', flat=True))

    def ordered_skus(self, *fields):
        return list(Battery.objects.order_by(*fields).values_list('sku', flat=True))

    def test_product_list_uses_expected_template(self):
        response = self.client.get(reverse('product_list'))

        self.assertTemplateUsed(response, 'catalog/product_list.html')

    def test_product_list_context_contains_products_and_types(self):
        response = self.client.get(reverse('product_list'))

        self.assertIn('products', response.context)
        self.assertIn('battery_types', response.context)
        self.assertEqual(response.context['products'].count(), 4)
        self.assertEqual(response.context['battery_types'].count(), 4)

    def test_product_list_renders_products_from_database(self):
        response = self.client.get(reverse('product_list'))

        self.assertContains(response, 'VARTA Blue Dynamic D24')
        self.assertContains(response, 'Xiaomi Redmi Power Bank 20000')

    def test_search_filters_products_by_query(self):
        response = self.client.get(reverse('product_list'), {'q': 'VARTA'})

        self.assertContains(response, 'VARTA Blue Dynamic D24')
        self.assertNotContains(response, 'Xiaomi Redmi Power Bank 20000')
        self.assertEqual(self.product_skus(response), ['AKB-VARTA-D24'])

    def test_type_filter_shows_only_selected_group(self):
        response = self.client.get(reverse('product_list'), {'type': 'aa-aaa'})

        self.assertContains(response, 'GP ReCyko AA 2500')
        self.assertNotContains(response, 'VARTA Blue Dynamic D24')
        self.assertEqual(self.product_skus(response), ['AKB-GP-AA-2500'])

    def test_availability_filter_shows_out_of_stock_products(self):
        response = self.client.get(reverse('product_list'), {'availability': 'out_of_stock'})

        self.assertEqual(self.product_skus(response), ['AKB-GP-AA-2500'])
        self.assertContains(response, 'Нет в наличии')

    def test_empty_product_list_renders_empty_state(self):
        response = self.client.get(reverse('product_list'), {'q': 'нет-такого-товара'})

        self.assertEqual(response.context['products'].count(), 0)
        self.assertContains(response, 'По заданным условиям аккумуляторы не найдены.')

    def test_product_list_sorts_by_name_by_default(self):
        response = self.client.get(reverse('product_list'))

        self.assertEqual(response.context['selected_sort'], 'name')
        self.assertEqual(self.product_skus(response), self.ordered_skus('name'))

    def test_product_list_sorts_by_required_price_field(self):
        response = self.client.get(reverse('product_list'), {'sort': 'price'})

        self.assertEqual(response.context['selected_sort'], 'price')
        self.assertEqual(self.product_skus(response), self.ordered_skus('price', 'name'))

    def test_product_list_sorts_by_optional_weight_field(self):
        response = self.client.get(reverse('product_list'), {'sort': 'weight'})

        self.assertEqual(response.context['selected_sort'], 'weight')
        self.assertEqual(self.product_skus(response), self.ordered_skus('weight_grams', 'name'))

```

```

def test_product_list_sorts_by_type_field(self):
    response = self.client.get(reverse('product_list'), {'sort': 'type'})

    self.assertEqual(response.context['selected_sort'], 'type')
    self.assertEqual(self.product_skus(response), self.ordered_skus('battery_type__name',
'name'))

def test_unknown_sort_falls_back_to_name(self):
    response = self.client.get(reverse('product_list'), {'sort': 'wrong'})

    self.assertEqual(response.context['selected_sort'], 'name')
    self.assertEqual(self.product_skus(response), self.ordered_skus('name'))

def test_filters_and_sort_can_be_combined(self):
    response = self.client.get(
        reverse('product_list'),
        {'availability': 'in_stock', 'sort': 'price'},
    )

    self.assertEqual(
        self.product_skus(response),
        ['AKB-XIAOMI-20000', 'AKB-CSB-HRL1234', 'AKB-VARTA-D24'],
    )

def test_product_detail_uses_expected_template_and_context(self):
    product = Battery.objects.get(sku='AKB-VARTA-D24')

    response = self.client.get(reverse('product_detail', args=[product.pk]))

    self.assertTemplateUsed(response, 'catalog/product_detail.html')
    self.assertEqual(response.context['product'], product)
    self.assertContains(response, product.description)
    self.assertContains(response, '540 A')

def test_about_uses_expected_template_and_text(self):
    response = self.client.get(reverse('about'))

    self.assertTemplateUsed(response, 'catalog/about.html')
    self.assertContains(response, 'Магазин специализируется на аккумуляторах')

```

ВЫВОДЫ

В результате выполнения практической работы был доработан проект каталога аккумуляторов на Django. Главная страница каталога теперь поддерживает поиск, фильтрацию по типу, фильтрацию по наличию и сортировку по нескольким полям.

В представлении `product_list` реализована обработка GET-параметра `sort`. Каталог может сортировать товары по названию, цене, весу и типу аккумулятора. Для неверного значения сортировки предусмотрен возврат к варианту по умолчанию, что повышает устойчивость страницы к некорректным запросам.

Тесты были перенесены в пакет `catalog/tests` и разделены по назначению. Файл `test_routes.py` проверяет маршруты, `reverse`, `resolve`, GET-статусы и 404. Файл `test_content.py` проверяет шаблоны, контекст, вывод данных, поиск, фильтрацию, сортировку и страницу `About`.

Всего выполнено 23 автоматизированных теста, все тесты прошли успешно. Покрытие файла `catalog/views.py` составляет 100%, а общее покрытие проекта по отчету `coverage` составляет 94%.

Таким образом, цель практической работы достигнута.